INSTITUTO SUPERIOR DE FORMACIÓN DOCENTE Y TÉCNICA N.º 57

Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial

INFORME FINAL INTEGRADOR

Sistema Inteligente de Predicción de Demanda de Variedades y Planificación Logística del Banco de Semillas

ProHuerta · Chascomús, Buenos Aires

Modelos de Aprendizaje Automático · Grupo 3



Huerta comunitaria. Imagen de referencia del proyecto ProHuerta Chascomús.

Integrantes	Barreu, Lucas · Quiroga, Franco · Pérez Cazal, Marcelo
Docente	[Completar]
Comisión	[Completar]
Fecha de entrega	3 de noviembre de 2026
Versión del informe	1.0
Aplicación web (demo)	https://pro-huerta.vercel.app/
Repositorio	github.com/lucasbar28/ProHuerta

Resumen ejecutivo

El presente informe documenta el desarrollo, la evaluación y el despliegue de un sistema de apoyo a la planificación logística para el Banco de Intercambio de Semillas del programa ProHuerta en Chascomús. El problema central es la incertidumbre sobre la cantidad y diversidad de variedades que solicitará cada persona al retirar su kit de semillas: preparar de más inmoviliza insumos limitados y preparar de menos genera faltantes, demoras y una experiencia deficiente para las familias.

La solución construida integra un pipeline reproducible de preparación de datos, modelos supervisados de regresión y clasificación, reglas de negocio explícitas y una aplicación web Flask con visualización territorial en 3D. El sistema se alimenta de **2.730 solicitudes** históricas correspondientes a siete campañas estacionales (2022–2026). Para el modelado se utilizan las **1.487 solicitudes específicas** con variedades observadas entre 2024 y 2026, evitando el sesgo de los kits genéricos de 2022–2023.

El modelo principal de demanda es un **KNeighborsRegressor (k = 41)** que estima los sobres equivalentes por solicitud a partir del perfil socio-productivo y del histórico barrial. La evaluación final sobre el conjunto de prueba calibrado arroja **RMSE = 3,39** y **MAE = 2,60 sobres**. En paralelo, un **árbol de decisión** estima la alerta de alta demanda (cobertura del catálogo ≥ 75 %) y un **modelo lineal** purgado evalúa el efecto de las precipitaciones sobre la cobertura. La aplicación expone estos modelos mediante una API REST y cinco vistas: portada, inicio, mapa de demanda, control de stock, análisis climático e informe de datos.

Resultado principal. El sistema no pretende predecir con exactitud el pedido de una familia individual; su valor está en la **planificación agregada por barrio y temporada**, en la detección temprana de sobredemanda y en la trazabilidad metodológica. La honestidad de las métricas y la documentación de las limitaciones forman parte del diseño y de la evaluación crítica del trabajo.

Palabras clave: aprendizaje automático, ProHuerta, predicción de demanda, planificación logística, regresión, clasificación, visualización geoespacial, Flask, scikit-learn.

Índice de contenidos

Resumen ejecutivo 2

1. Introducción 5

1.1. Contexto institucional y territorial . . . 5
1.2. Problemática . . . 5
1.3. Objetivos . . . 5
1.4. Alcance y usuarios . . . 5
1.5. Organización del documento . . . 6

2. Datos 7

2.1. Fuentes y volumetría . . . 7
2.2. Esquemas de entrega: kits genéricos y específicos . . . 7
2.3. Auditoría y control de calidad . . . 7
2.4. Diccionario resumido de variables . . . 8
2.5. Análisis exploratorio . . . 9
2.6. Limitaciones de los datos . . . 9

3. Metodología y preparación de datos 10

3.1. Arquitectura del pipeline . . . 10
3.2. Limpieza, normalización e ingeniería de variables . . . 10
3.3. Partición y estratificación . . . 10
3.4. Prevención de fuga de información (data leakage) . . . 10

4. Modelado 12

4.1. Modelo de regresión de demanda . . . 12
4.1.1. Variable objetivo y predictores . . . 12
4.1.2. Comparación de algoritmos . . . 12
4.1.3. Calibración estacional . . . 12
4.1.4. Métricas finales . . . 13
4.1.5. Backtest temporal . . . 13
4.2. Modelo de clasificación y alerta de alta demanda . . . 13
4.2.1. Variable objetivo y umbral . . . 13
4.2.2. Esquema de costos . . . 14
4.3. Modelo climático . . . 14
4.4. Regla de alerta territorial . . . 14

5. Arquitectura de la aplicación 15

5.1. Visión general . . . 15
5.2. Estructura del repositorio . . . 15
5.3. Módulos y vistas . . . 15
Mapa de demanda . . . 16
Control de stock . . . 16
Análisis climático . . . 16
5.4. API REST . . . 16
5.5. Frontend y visualización . . . 16

5.6. Despliegue y ejecución local 17
6. Resultados y discusión **18**
6.1. Desempeño del modelo de regresión 18
6.2. Desempeño del clasificador y del sistema de alertas 18
6.3. Interpretación operativa 18
6.4. Limitaciones y advertencias 18
6.5. Aportes del proyecto 19
7. Conclusiones **20**
8. Trabajo futuro **21**
9. Referencias **22**
Anexos **23**
Anexo A. Estructura de archivos generados 23
Anexo B. Reproducibilidad 23
Anexo C. Glosario 23

1. Introducción

1.1. Contexto institucional y territorial

El programa ProHuerta, impulsado por el INTA y articulado con municipios y organizaciones locales, promueve la producción agroecológica de alimentos y el acceso a semillas de calidad. En Chascomús, el Banco de Intercambio de Semillas cumple un rol central en la entrega de kits estacionales de semillas a huertas familiares, escolares, comunitarias e institucionales. Cada campaña se organiza en dos temporadas: **Otoño–Invierno (OI)** y **Primavera–Verano (PV)**.

La diversidad de variedades que solicita cada participante depende de múltiples factores: el tipo de huerta, la superficie disponible, la experiencia previa, el destino de la producción, la ocupación de la persona responsable y el barrio donde se localiza la huerta. Esa heterogeneidad dificulta la preparación anticipada de kits completos y la asignación presupuestaria.

1.2. Problemática

El equipo logístico debe decidir, antes de la entrega, si prepara kits completos o kits básicos estandarizados. Si prepara de más, inmoviliza sobres que podrían haberse destinado a otras familias; si prepara de menos, faltan variedades en el mostrador y se afecta la diversidad de la huerta y la experiencia del programa. El problema es, en esencia, un problema de **predicción de demanda y planificación bajo incertidumbre**.

1.3. Objetivos

Objetivo general. Diseñar e implementar un sistema de aprendizaje automático que anticipe la demanda de variedades de semillas y aporte información para la planificación logística, presupuestaria y territorial del Banco de Semillas de Chascomús.

Objetivos específicos.

1. Consolidar y auditar las bases históricas de solicitudes de semillas de las temporadas 2022–2026, resolviendo inconsistencias de formato, barrio y especies.
2. Construir un dataset modelable con variables socio-productivas disponibles al momento de la inscripción, evitando fugas de información.
3. Entrenar y comparar modelos de regresión para estimar la cantidad de sobres equivalentes por solicitud.
4. Entrenar un clasificador de alta demanda y definir reglas de negocio para la activación de alertas logísticas.
5. Desarrollar una aplicación web que integre los modelos, un mapa territorial en 3D y módulos de simulación de stock y clima.
6. Documentar métricas, supuestos, limitaciones y reproducibilidad del sistema.

1.4. Alcance y usuarios

El sistema está orientado a coordinadores logísticos, promotores territoriales y referentes de la Dirección de Ambiente y Desarrollo Sustentable. Se utiliza como **apoyo a la decisión**, no como mecanismo automático de asignación de insumos. La unidad de análisis es la solicitud individual de retiro de semillas; la unidad de planificación es el barrio y la temporada.

1.5. Organización del documento

El documento se organiza en ocho secciones y cuatro anexos. Las secciones 2 y 3 describen los datos y la metodología; la sección 4 presenta los modelos y sus métricas; la sección 5 detalla la arquitectura de la aplicación; la sección 6 discute los resultados; las secciones 7 y 8 sintetizan conclusiones y trabajo futuro. Los anexos contienen la estructura del repositorio, los endpoints de la API, los comandos de reproducción y un glosario.

2. Datos

2.1. Fuentes y volumetría

La información proviene de los formularios de retiro de semillas del Banco de Intercambio de Semillas de Chascomús. El script **preparar_datos.py** consolida siete archivos Excel correspondientes a siete campañas estacionales, entre 2022 y 2026. La Tabla 1 resume las fuentes y la cantidad de filas procesadas.

Año	Temp.	Archivo de origen	Hoja	Filas
2022	OI	2022 registro de huerta otoño-invierno 2022.xlsx	general	488
2023	OI	2023 Relevamiento otoño invierno para filtrar.xlsx	DATOS	750
2024	PV	2024 retiro de semillas prim-ver. 2024. + seguimiento + devolución.xlsx	familiares	273
2025	OI	2025 RETIRO de semillas 2025 por primera vez OI 2025 (respuestas).xlsx	Respuestas de formulario 1	295
2025	PV	2025 retiro de semillas por primera vez PV 2025 (respuestas).xlsx	Respuestas de formulario 1	464
2026	OI	2026 Retiro de semillas por primera vez O-I 2026 (respuestas).xlsx	Respuestas de formulario 1	237
2026	PV	2026 Retiro de semillas P-V 2026 (respuestas).xlsx	Respuestas de formulario 1	223

Tabla 1. Fuentes de datos y filas procesadas por campaña. Total: 2.730 solicitudes.

2.2. Esquemas de entrega: kits genéricos y específicos

El histórico presenta dos esquemas de entrega. Entre 2022 y 2023 se entregaba un **kit genérico de diez especies surtidas**, sin detalle de variedades en el formulario. A partir de 2024 se registra la **selección específica de variedades**, lo que permite observar la demanda real de cada solicitud. Esta diferencia es central: los kits genéricos solo pueden imputarse mediante una canasta de referencia, mientras que las solicitudes específicas contienen el target observado.

Esquema	Período	Filas	Cobertura media	Alerta media	Sobres equivalentes medios	Presupuesto total
Específico	2024–2026	1.492	62,84 %	0,409	7,74	$ 5.754.500
Kit genérico	2022–2023	1.238	N/A (imputada)	N/A	13,47	$ 7.040.000

Tabla 2. Distribución por esquema de entrega. El presupuesto se calcula con un valor fijo de $500 por sobre.

> **Decisión metodológica.** Para entrenar los modelos se utilizan únicamente las **1.487 solicitudes específicas con especies observadas**. Los kits genéricos se conservan en el dataset para auditoría y análisis de volumen, pero no se usan como target de los modelos principales, porque su contenido de variedades es una imputación y no un dato observado.

2.3. Auditoría y control de calidad

El proceso de preparación aplica controles de reconciliación sobre todas las filas. La Tabla 3 resume los principales indicadores de calidad. No se eliminó ninguna fila: los problemas se documentan mediante banderas de calidad (*flags*) para que puedan auditarse.

Control técnico	Resultado
Solicitudes totales validadas	2.730

Control técnico	Resultado
Registros 2022–2023 sin dato explícito de kits	193
Registros 2022–2023 con 0 kits reales	8
Registros 2024–2026 sin texto descriptivo de especies	5
Registros 2024–2026 con conteo de 0 especies	1
Solicitudes sin asignación de barrio (sin_dato)	786
Solicitudes con geolocalización (coordenadas)	969
Solicitudes genéricas con demanda imputada por canasta	1.037
Sobres equivalentes totales imputados (kits × 10)	14.080

Tabla 3. Controles de calidad y reconciliación del dataset consolidado.

Los registros sin barrio se concentran en 2024 y en la temporada OI 2025, cuyos formularios no incluían la pregunta de barrio. Esa limitación se explicita en el mapa: la proyección futura solo cubre los **31 barrios mapeados con datos**, mientras que el total municipal observado incluye los registros sin barrio y los barrios fuera de la dimensión geográfica.

2.4. Diccionario resumido de variables

El dataset consolidado **solicitudes_v2.csv** contiene una fila por solicitud. La Tabla 4 presenta las variables principales. El diccionario completo se encuentra en **informe/diccionario_datos.md**.

Variable	Tipo	Descripción
id_solicitud	texto	Identificador único: año-temporada-fila.
anio / temporada	entero / texto	Campaña (2022–2026) y temporada OI o PV.
esquema_entrega	texto	kit_generico (2022–2023) o especifico (2024–2026).
barrio_norm / barrio_nombre	texto	Barrio normalizado y nombre para mostrar.
tipo_huerta	texto	familiar, institucional o no especificado.
tipo_localizacion	texto	urbana, periurbana, rural o no especificado.
experiencia	texto	primera vez, 1 a 3 años, más de 3 años o no especificado.
superficie	texto	hasta 5 m², entre 6 y 20 m², más de 20 m² o no especificado.
destino	texto	autoconsumo, autoconsumo e intercambio, venta / excedentes.
ocupacion	texto	Ocupación agrupada del responsable de la huerta.
kits	decimal	Kits entregados en 2022–2023; nulo si no hay dato.
cantidad_variedades	decimal	Especies distintas solicitadas en 2024–2026.
especies_equivalentes	decimal	kits × 10 (genérico) o cantidad_variedades (específico).
oferta_catalogo	entero	Especies ofrecidas en el catálogo de la temporada.
cobertura_catalogo	decimal	min(100, especies_equivalentes / oferta × 100).
alerta_alta_demanda	entero	1 si cobertura ≥ 75 %, 0 en caso contrario.
presupuesto_estimado	decimal	especies_equivalentes × 500.
flags_calidad	texto	Banderas de calidad separadas por punto y coma.

Tabla 4. Diccionario resumido de variables del dataset consolidado.

2.5. Análisis exploratorio

El análisis exploratorio muestra que la demanda de variedades no es uniforme: un grupo reducido de especies concentra la mayor parte de las solicitudes. La Tabla 5 presenta el top diez de especies observadas en las temporadas OI 2025 y OI 2026, utilizado para construir la canasta de referencia de los kits genéricos.

#	Especie	Solicitudes	% de solicitudes	En canasta top 10
1	Perejil	451	85,1 %	Sí
2	Acelga	441	83,2 %	Sí
3	Lechuga	429	80,9 %	Sí
4	Cebolla	348	65,7 %	Sí
5	Remolacha	345	65,1 %	Sí
6	Espinaca	340	64,2 %	Sí
7	Flor	249	47,0 %	Sí
8	Rabanito	247	46,6 %	Sí
9	Habas	237	44,7 %	Sí
10	Rúcula	155	29,2 %	Sí

Tabla 5. Top 10 de especies observadas en OI 2025–2026 (canasta OI_2025_2026_v1).

La distribución por temporada y año (Tabla 1) evidencia un crecimiento de la oferta de catálogo: pasó de 9 variedades en 2024 PV a 18 en 2026 PV. Esa variación justifica normalizar la demanda como **cobertura porcentual del catálogo** y no solo como cantidad absoluta de variedades. También explica por qué el modelo principal incorpora el año y la oferta como variables de contexto.

2.6. Limitaciones de los datos

- Los formularios de 2024 y OI 2025 no registran barrio; por eso una parte importante de las solicitudes queda como **sin_dato** y no puede asignarse al mapa.
- Los kits genéricos de 2022–2023 no detallan variedades: su composición es una imputación basada en la canasta OI 2025–2026, con confianza baja.
- La serie histórica es corta (siete campañas) y presenta cambios de formulario entre años, lo que limita la comparabilidad estricta.
- El precio de $500 por sobre es un valor fijo de referencia, no un precio de mercado.

3. Metodología y preparación de datos

3.1. Arquitectura del pipeline

El flujo de trabajo separa estrictamente la fase **offline** de preparación y entrenamiento de la fase **online** de inferencia. La aplicación web no entrena modelos en tiempo de ejecución: carga artefactos serializados y ejecuta únicamente *predict*. Esta separación garantiza tiempos de respuesta bajos, reproducibilidad y trazabilidad.

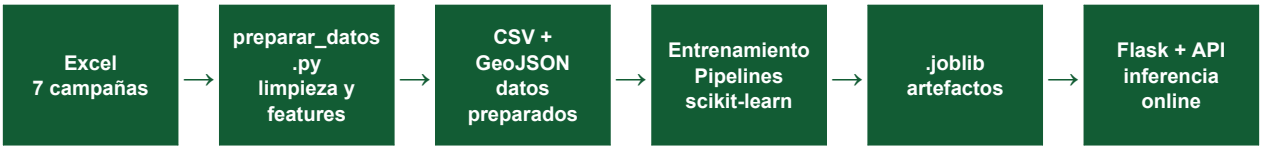


Figura 1. Flujo general del proyecto: de los formularios Excel a la aplicación web.

3.2. Limpieza, normalización e ingeniería de variables

El script **preparar_datos.py** implementa un proceso de limpieza robusto que incluye corrección de codificación (*mojibake*), normalización de textos, detección automática de columnas por patrones y mapeo de categorías. Las principales transformaciones son:

- **Barrios:** normalización a una clave canónica, eliminación de prefijos (*barrio*, *B°*), resolución de alias y errores de escritura, y asignación de **sin_dato** cuando no hay información.
- **Especies:** tokenización por punto y coma, coma o salto de línea, eliminación de tokens inválidos (*otra*, *ninguna*, etc.) y canonicalización a un catálogo controlado de especies.
- **Experiencia:** estandarización en tres niveles: primera vez, 1 a 3 años y más de 3 años.
- **Superficie:** conversión de rangos y textos heterogéneos a tres categorías (hasta 5 m², entre 6 y 20 m², más de 20 m²).
- **Tipo de huerta:** consolidación en familiar, institucional o no especificado.
- **Destino y ocupación:** agrupamiento de respuestas abiertas en categorías comparables.

La ingeniería de variables del modelo principal incorpora, además del perfil socio-productivo, señales históricas agregadas por barrio y temporada: media histórica de sobres, cantidad de solicitudes, cantidad de años con datos, ratio del barrio frente a la temporada y ratio del perfil frente al barrio. Estas variables permiten que el modelo combine información individual con contexto territorial.

3.3. Partición y estratificación

Para el modelo principal, el conjunto de **1.487 solicitudes específicas** se dividió en entrenamiento, validación y prueba con una proporción 60/20/20. La partición se estratificó por temporada, año y tercil de demanda, con **random_state = 42**, para preservar la proporción de clases y reducir el riesgo de fugas interanuales.

Conjunto	Observaciones	Uso
Entrenamiento	892	Ajuste de codificadores, escaladores y modelos.
Validación	297	Selección de hiperparámetros, umbrales y calibración.
Prueba	298	Evaluación final, reservada a ciegas.

Tabla 6. Partición del dataset modelado (60/20/20).

3.4. Prevención de fuga de información (data leakage)

Se aplicaron tres controles para evitar que el modelo aprenda información que no estará disponible al momento de la predicción:

1. Se excluyeron del conjunto de predictores las variables derivadas del target: **cobertura_catalogo**, **alerta_alta_demanda** y **presupuesto_estimado**.
2. Se excluyó **cantidad_variedades** bruta, que es el insumo directo del target de regresión, y **kits**, que solo existe en la era genérica.
3. Todo el preprocesamiento se encapsuló en **Pipelines** y **ColumnTransformer** de scikit-learn, ajustados únicamente sobre el conjunto de entrenamiento.

En el módulo climático se detectó y corrigió un caso adicional de fuga: la versión preliminar usaba **Oferta_Catalogo** para predecir la cobertura, pero la oferta está matemáticamente relacionada con el target. El modelo final se reentrenó usando exclusivamente **Precipitaciones_Hist_mm**.

4. Modelado

El proyecto implementa dos familias de modelos complementarias. La primera trabaja a nivel de **solicitud** y estima la cantidad de sobres equivalentes; se usa para el mapa territorial y la planificación agregada. La segunda trabaja a nivel de **perfil de huerta** y estima la cobertura del catálogo y la alerta de alta demanda; se usa en el módulo de control de stock. A ellas se suma un modelo climático simple para el módulo de lluvias.

> **Unidad de medida.** Un **sobre equivalente** representa una variedad solicitada. En 2022–2023 se calcula como *kits* × *10*; en 2024–2026 coincide con la cantidad de variedades observadas. El valor de referencia presupuestario es de **$500 por sobre**.

4.1. Modelo de regresión de demanda

4.1.1. Variable objetivo y predictores

El target es **especies_equivalentes**. Sus estadísticas descriptivas sobre el conjunto modelado son: media 7,74; mediana 8; desvío estándar 3,75; mínimo 0 y máximo 29. El modelo recibe trece predictores: año indexado, temporada, tipo de huerta, tipo de localización, experiencia, superficie, destino, ocupación, media histórica del barrio, cantidad histórica de solicitudes, cantidad de años históricos, ratio barrio/temporada y ratio perfil/barrio.

Las variables se procesan mediante un **ColumnTransformer**: las numéricas se estandarizan con **StandardScaler** y las categóricas se codifican con **OneHotEncoder**. Esto es especialmente importante para KNN, que depende de distancias entre observaciones.

4.1.2. Comparación de algoritmos

Se compararon distintos algoritmos y configuraciones sobre el conjunto de validación. La Tabla 7 muestra los cinco mejores resultados por RMSE. El **KNeighborsRegressor con k = 41** obtuvo el menor error y fue seleccionado.

Algoritmo e hiperparámetros	RMSE	MAE	R^2
KNeighborsRegressor (k=41, weights=uniform)	3,543	2,669	0,1957
KNeighborsRegressor (k=21, weights=uniform)	3,583	2,693	0,1775
Ridge (alpha=1000.0)	3,605	2,706	0,1669
RandomForestRegressor (depth=6, leaf=3)	3,626	2,699	0,1575
RandomForestRegressor (depth=4, leaf=5)	3,635	2,718	0,1532

Tabla 7. Comparación de algoritmos de regresión en validación (fuente: comparacion_regresion.csv).

4.1.3. Calibración estacional

Las predicciones crudas presentaban un sesgo sistemático por temporada. Se aplicó un factor de calibración multiplicativo: **1,0815 para OI** y **1,0745 para PV**. La calibración mejora levemente el RMSE y el MAE de validación, aunque no corrige la baja varianza explicada a nivel individual.

4.1.4. Métricas finales

Conjunto	RMSE	MAE	R²
Entrenamiento	2,793	2,119	0,4466
Validación (cruda)	3,543	2,669	0,1957
Validación (calibrada)	3,498	2,652	0,2158
Prueba (cruda)	3,327	2,574	0,0999
Prueba (calibrada)	3,393	2,602	0,0644

Tabla 8. Métricas de regresión por conjunto. La validación cruzada de 5 folds arroja RMSE = 2,865.

4.1.5. Backtest temporal

Para evaluar la capacidad de generalización temporal se entrenó con 2024–2025 y se evaluó en 2026. El modelo obtuvo un RMSE de 5,296 frente a 5,500 del baseline ingenuo, un MAE de 4,006 frente a 4,073 y un R² de −0,131. La mejora es marginal: el modelo apenas supera al baseline y no captura de forma robusta la variación interanual.

Métrica	Modelo	Baseline ingenuo
RMSE	5,296	5,500
MAE	4,006	4,073
R²	−0,131	—

Tabla 9. Backtest temporal: entrenamiento 2024–2025, evaluación 2026.

> **Lectura crítica.** El MAE de prueba (2,60 sobres) representa aproximadamente el 33,6 % de la media del target (7,74). El R² de prueba es bajo (0,064) y la brecha entrenamiento–prueba es amplia (0,382). En consecuencia, el modelo **no es adecuado para decisiones individuales**: se utiliza para estimar totales agregados por barrio y temporada, donde los errores tienden a compensarse.

4.2. Modelo de clasificación y alerta de alta demanda

4.2.1. Variable objetivo y umbral

La variable binaria **alerta_alta_demanda** vale 1 cuando la cobertura del catálogo es igual o superior al **75 %**, es decir, cuando la solicitud pide al menos tres de cada cuatro variedades disponibles. Ese umbral es el punto de inflexión operativo a partir del cual conviene preparar anticipadamente un kit casi completo. La cobertura se define como:

```
cobertura_catalogo = min(100, especies_equivalentes / oferta_catalogo × 100)
```

Se evaluaron distintas configuraciones de árboles de decisión, con y sin balanceo de clases, y distintos umbrales de decisión. La Tabla 10 resume la mejor configuración encontrada y el efecto del ajuste de umbral.

Configuración	Precision	Recall	F1	F2	Costo
Árbol depth=5, hoja=1, entropía (umbral 0,50)	0,531	0,779	0,631	0,712	138
Árbol depth=5, hoja=1, entropía (umbral 0,37)	0,517	0,877	0,650	0,770	130

Tabla 10. Evaluación del clasificador de alta demanda (fuente: comparacion_clasificacion.csv).

El ajuste del umbral a 0,37 reduce los falsos negativos de 27 a 15 y baja el costo total de 138 a 130, a costa de aumentar los falsos positivos de 84 a 100. Esta decisión es coherente con el esquema de costos adoptado.

4.2.2. Esquema de costos

El proyecto asigna un costo social y operativo asimétrico a los errores del clasificador:

- **Falso positivo (FP):** se predice alta demanda, pero la persona solicita un kit acotado. Se preparó un kit completo de más; los sobres no retirados se reasignan. Costo asignado: **1 unidad**.
- **Falso negativo (FN):** se predice demanda estándar, pero la persona necesitaba un kit completo. Faltan variedades en el mostrador y se afecta la diversidad de la huerta. Costo asignado: **2 unidades**.

```
Costo total = 1 × FP + 2 × FN
```

Por eso los hiperparámetros y umbrales se ajustaron priorizando el **Recall** y el **F2-Score** por sobre la precisión. En el módulo de stock se muestran dos sistemas independientes —regresión con umbral y clasificador directo— para que el usuario detecte casos dudosos y, ante una discrepancia, opte por preparar el kit completo.

4.3. Modelo climático

El módulo de análisis climático estima cómo varía la cobertura del catálogo según las precipitaciones históricas. El modelo final es una **regresión lineal simple** entrenada únicamente con **Precipitaciones_Hist_mm**, luego de detectar y eliminar una fuga de información asociada a **Oferta_Catalogo**.

```
cobertura_estimada (%) = 55,4973 + 0,013267 × precipitaciones (mm)
```

El coeficiente indica que cada 100 mm adicionales de lluvia se asocian con un aumento de aproximadamente **1,33 puntos porcentuales** en la cobertura estimada. La salida se acota lógicamente entre 0 % y 100 %. La interfaz clasifica el escenario en sequía, régimen promedio o precipitaciones intensas según la temporada y los milímetros ingresados.

4.4. Regla de alerta territorial

Para el mapa de demanda, la alerta no se dispara con el clasificador individual sino con una regla de negocio sobre la proyección agregada por barrio. Un barrio queda en alerta cuando la demanda proyectada supera el baseline histórico y se cumplen dos condiciones simultáneas:

```
predicho > esperado × (1 + 0,10) Y (predicho − esperado) ≥ 10 sobres
```

El doble criterio evita falsas alarmas en barrios pequeños, donde una variación porcentual grande puede originarse en una base mínima. El baseline se calcula como la media anual histórica de sobres por barrio y temporada, ajustada por un crecimiento anual supuesto del 5 %:

```
esperado(año) = media_anual_sobres × (1 + 0,05)^(año − último_año)
```

Los barrios con menos de **10 solicitudes históricas** o menos de **2 años de datos** se marcan como **muestra limitada**, y la alerta se acompaña de una advertencia de cautela. La proyección se calcula para 2027 y 2028, repitiendo la última oferta de catálogo observada y aplicando el crecimiento supuesto. Se trata de **escenarios base**, no de pronósticos validados.

5. Arquitectura de la aplicación

5.1. Visión general

La aplicación es una **aplicación web Flask** con renderizado del lado del servidor mediante plantillas Jinja2, estilos CSS propios y JavaScript para las visualizaciones interactivas. Los modelos se cargan desde archivos **.joblib** al iniciar el proceso; las consultas se responden con inferencia en memoria y sin base de datos. El despliegue en producción se realiza en **Vercel** mediante el runtime **@vercel/python**.

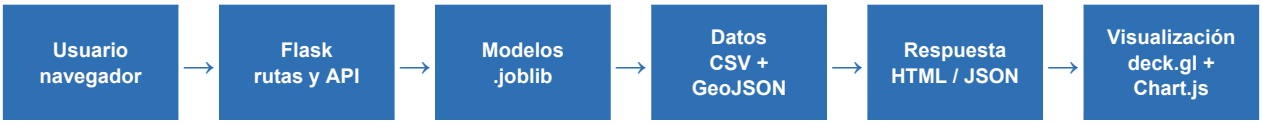


Figura 2. Arquitectura lógica de la aplicación y flujo de una consulta.

5.2. Estructura del repositorio

Elemento	Contenido
app.py	Aplicación Flask principal: rutas, API, carga de modelos y lógica de agregación territorial.
dash_app.py	Prototipo alternativo de tablero en Dash (puerto 8050). No es necesario para el despliegue principal.
preparar_datos.py	Pipeline de consolidación, limpieza, normalización y generación de CSVs auditados.
logica_demanda.py	Reglas de negocio de variación, alertas y calidad de muestra, independientes de Flask.
logica_lluvias.py	Carga y uso del modelo climático purgado.
entrenar_guardar_modelos.py	Entrenamiento de los modelos de cobertura y clasificación del módulo de stock.
act_lluvia.py / entrenar_modelo_lluvias.py	Reentrenamiento del modelo climático; la versión vigente es la purgada.
modelos/	Artefactos .joblib y metadata_modelos.json.
templates/	Vistas Jinja2: portada, inicio, mapa, inventario, lluvias e informe.
static/	CSS, JavaScript, imágenes y basemap de Chascomús.
datos/preparados/	CSVs consolidados, especies, catálogo, canasta y dimensión de barrios.
datos/geospatial/	GeoJSON de barrios, radios censales y extensiones territoriales.
informe/	Auditoría, diccionario, comparación de modelos y predicciones de test.

Tabla 11. Estructura principal del repositorio.

5.3. Módulos y vistas

La navegación se unifica mediante un diccionario centralizado **NAV_URLS** inyectado por un procesador de contexto de Flask. La Tabla 12 describe las seis vistas principales.

Vista	Ruta	Función principal
Portada	/	Presentación institucional, iniciativa, objetivos y acceso al sistema.
Inicio	/inicio	Panel de acceso a herramientas y KPIs reales del proyecto.
Mapa de demanda	/mapa	Mapa 3D por barrio, comparativa, tendencia y alertas de sobredemanda.
Control de stock	/inventario	Simulación de cobertura por perfil de huerta y doble alerta.
Análisis climático	/lluvias	Estimación del impacto de las precipitaciones en la cobertura.
Informe de datos	/informe	Documentación técnica, variables, métricas y limitaciones.

Tabla 12. Vistas principales de la aplicación.

Mapa de demanda

El mapa utiliza **deck.gl** para renderizar un basemap satelital de Chascomús, los polígonos de los barrios y columnas 3D extruidas que comparan la demanda de referencia con la proyectada. El panel lateral permite filtrar por **temporada, año, variable** (sobres, solicitudes o presupuesto) y **barrio**. Al seleccionar un barrio se muestra su detalle: valores de referencia y proyección, variación porcentual, estado de alerta, muestra histórica, superficie, radios censales y especies más solicitadas. Debajo se presentan una comparativa por barrio (Chart.js) y una serie de tendencia con datos observados, baseline y proyección.

Control de stock

El formulario de control de stock recibe el perfil de una solicitud (temporada, año, localización, tipo de huerta, experiencia, superficie, destino y ocupación) y estima la cobertura porcentual del catálogo. A partir de esa estimación se muestran dos alertas: la regresión con umbral (≥ 75 %) y el clasificador directo. Si ambos coinciden, la interfaz lo indica; si discrepan, recomienda preparar el kit completo porque el falso negativo cuesta el doble. El módulo usa los modelos **modelo_regresion_cobertura.joblib** y **modelo_clasificacion_dash.joblib**.

Análisis climático

El módulo de lluvias recibe temporada, año y milímetros esperados. A partir de esos datos estima la cobertura con el modelo lineal purgado y presenta un semáforo de impacto: sequía, régimen promedio o precipitaciones intensas. El resultado se acota entre 0 % y 100 % y se contextualiza con los promedios históricos de 650 mm para PV y 380 mm para OI.

5.4. API REST

El backend expone una API JSON que alimenta las vistas y permite integraciones futuras. La Tabla 13 resume los endpoints principales.

Método	Ruta	Descripción
GET	/api/health	Estado del servicio, versión del modelo, filas, barrios, años de proyección y umbrales.
GET	/api/filtros	Períodos disponibles, barrios mapeados, margen de sobredemanda y precio del sobre.
GET	/api/mapa_completo?anio=&temporada;=	Datos agregados por barrio, series de ciudad, escenarios y resúmenes territorial y municipal.
GET	/api/consulta_especifica?barrio=&anio;=&temporada;=	Detalle de un barrio: métricas, geometría, centroide y especies.
POST	/api/prediccion	Predicción puntual de sobres equivalentes y presupuesto a partir de un perfil.

Tabla 13. Endpoints principales de la API Flask.

El endpoint **/api/mapa_completo** es el más complejo: integra la proyección por barrio, el baseline, las alertas, el resumen territorial (solo barrios mapeados con datos) y el resumen municipal observado (incluye registros sin barrio). Esta separación es explícita en la respuesta para evitar comparaciones engañosas entre universos distintos.

5.5. Frontend y visualización

La interfaz utiliza **Bootstrap 5** con el tema **Darkly**, íconos de Bootstrap Icons y la tipografía **Inter**. Los estilos propios se concentran en **static/css/prohuerta.css** y **portada.css**. La lógica de estado de los filtros se comparte en **static/js/common.js**, que persiste la selección en **localStorage** y en la querystring. Las visualizaciones del mapa se implementan en **static/js/mapa.js** con deck.gl y Chart.js. El diseño es responsivo y contempla navegación móvil.

Accesibilidad y transparencia. Las vistas incluyen etiquetas ARIA, textos alternativos y advertencias visibles sobre el alcance de las estimaciones. El mapa distingue visualmente referencia, proyección, estabilidad, aumento y alerta de sobredemanda.

5.6. Despliegue y ejecución local

El despliegue en producción se realiza en **Vercel**. El archivo **vercel.json** declara el runtime **@vercel/python** y redirige todas las rutas a **app.py**. Como la aplicación no entrena en vivo ni depende de una base de datos pesada, la inferencia se ejecuta en milisegundos y el entorno gratuito es suficiente para la demo académica. La aplicación está disponible en **https://pro-huerta.vercel.app/**.

Para ejecutarla localmente se requiere Python y las dependencias de **requirements.txt**. El comando **python app.py** inicia el servidor en **http://localhost:5000**. La Tabla 14 resume las dependencias principales.

Librería	Versión	Uso
Flask	3.0.3	Backend web, rutas y API.
scikit-learn	1.9.1	Pipelines, preprocesamiento, modelos y métricas.
pandas / numpy	2.2.3 / 2.1.2	Manipulación de datos y cálculo numérico.
joblib	1.4.2	Serialización y carga de modelos.
Dash / Plotly	≥ 2.17 / ≥ 5.22	Prototipo alternativo de tablero.
gunicorn	23.0.0	Servidor WSGI para despliegue.

Tabla 14. Dependencias principales del proyecto (requirements.txt).

6. Resultados y discusión

6.1. Desempeño del modelo de regresión

El modelo de regresión logra un ajuste aceptable en entrenamiento (R^2 = 0,447) pero pierde capacidad explicativa en validación (R^2 = 0,216 calibrado) y prueba (R^2 = 0,064). El MAE de prueba calibrado es de 2,60 sobres, sobre una media de 7,74. En términos relativos, el error ronda el 33,6 % del valor medio. El backtest temporal confirma la dificultad: el RMSE apenas mejora al baseline ingenuo (5,30 vs. 5,50) y el R^2 es negativo (−0,131).

Estos resultados no invalidan el sistema, pero delimitan su uso. La predicción individual es ruidosa porque la decisión de cada familia depende de factores no registrados en el formulario (preferencias, disponibilidad de semillas, clima, decisiones del momento). En cambio, al sumar muchas solicitudes por barrio y temporada, los errores individuales tienden a compensarse y la estimación agregada resulta más estable. Por eso el producto se diseña como una herramienta de **planificación territorial** y no como un predictor de casos puntuales.

6.2. Desempeño del clasificador y del sistema de alertas

El clasificador de alta demanda, ajustado a un umbral de 0,37, alcanza un **Recall de 0,877** y un **F2 de 0,770**, con un costo total de 130. La prioridad es detectar la mayor cantidad posible de solicitudes de alta demanda, aun a costa de un aumento de falsos positivos. En un contexto donde un falso negativo deja a una familia sin variedades y un falso positivo solo inmoviliza stock temporalmente, esta asimetría es razonable.

La convivencia de dos sistemas —regresión con umbral y clasificador directo— aporta transparencia: cuando ambos coinciden, la decisión es más confiable; cuando discrepan, el usuario recibe una advertencia y puede optar por la alternativa más conservadora. Esta doble lectura es una decisión de diseño orientada a la gestión del riesgo logístico.

6.3. Interpretación operativa

El sistema permite responder cuatro preguntas de gestión:

1. **¿Cuánto se espera demandar?** Mediante la proyección de sobres por barrio y temporada, con comparación contra el baseline histórico.
2. **¿Dónde se concentra el riesgo?** Mediante el mapa 3D, las alertas de sobredemanda y el listado de prioridades a reforzar.
3. **¿Qué perfil de huerta requiere un kit completo?** Mediante el módulo de stock y la cobertura estimada del catálogo.
4. **¿Cómo puede afectar el clima?** Mediante el módulo de lluvias y el escenario hídrico simulado.

6.4. Limitaciones y advertencias

El informe y la propia aplicación explicitan las siguientes limitaciones:

- Las proyecciones 2027–2028 repiten la oferta de catálogo de 2026 y aplican un crecimiento supuesto del 5 %: son **escenarios**, no pronósticos validados.
- El resumen territorial suma solo los barrios mapeados con datos; los registros sin barrio y los barrios fuera de la dimensión geográfica no se incluyen en la proyección futura.
- Los años 2024 y OI 2025 no tienen barrio en el formulario original; el baseline barrial usa solo los años con barrio.
- El precio de $500 por sobre es un valor fijo supuesto, no un precio de mercado.
- La alerta es una regla de decisión sobre la proyección, no una probabilidad calibrada.

- El desempeño individual del modelo es limitado; no debe usarse para decidir sobre una familia en particular.
- La muestra histórica es corta y los formularios cambian entre campañas, lo que limita la comparabilidad y la robustez de las series.

6.5. Aportes del proyecto

- **Integración completa:** datos, modelos, reglas de negocio, API, interfaz y despliegue en un solo sistema reproducible.
- **Auditoría y trazabilidad:** cada transformación y cada bandera de calidad queda documentada; el dataset conserva todas las filas.
- **Prevención de fuga de información:** exclusión de variables derivadas del target y corrección explícita del modelo climático.
- **Visualización territorial:** mapa 3D interactivo con comparación referencia vs. proyección y alertas por barrio.
- **Transparencia:** métricas honestas, limitaciones visibles y advertencias en la interfaz.

7. Conclusiones

El proyecto cumplió el objetivo de construir un flujo completo de aprendizaje automático para un problema real de planificación logística en el Banco de Semillas de Chascomús. Se consolidó un dataset de 2.730 solicitudes, se auditó su calidad, se construyeron modelos de regresión y clasificación con pipelines reproducibles, se definieron reglas de alerta explícitas y se desplegó una aplicación web con visualización territorial.

La principal enseñanza metodológica es que el valor de un modelo no depende solo de su métrica individual, sino de la coherencia entre el problema, la unidad de decisión y el uso previsto. El modelo de regresión no es preciso para anticipar el pedido de una familia, pero sí aporta información útil cuando se lo utiliza para estimar totales agregados por barrio y temporada. La decisión de diseñar el sistema como apoyo a la planificación —y no como asignador automático— es parte central del aprendizaje.

La segunda enseñanza es la importancia de la transparencia. El informe documenta las métricas reales, las limitaciones de la muestra, los supuestos de proyección y los riesgos de fuga de información. Esa honestidad no debilita el trabajo: lo vuelve defendible, reproducible y mejorable. El sistema queda preparado para incorporar nuevas campañas, reentrenar los modelos y ampliar la cobertura territorial a medida que los formularios registren el barrio de manera sistemática.

Conclusión final. El sistema transforma un conjunto disperso de formularios en información accionable para la planificación logística, presupuestaria y territorial, con un diseño metodológicamente riguroso y una interfaz pensada para usuarios no técnicos.

8. Trabajo futuro

A partir de las limitaciones identificadas, se proponen las siguientes líneas de mejora:

1. **Mejorar la captura de datos:** incorporar el barrio de manera obligatoria en todos los formularios y estandarizar las opciones de respuesta.
2. **Ampliar la serie histórica:** sumar nuevas campañas y evaluar modelos con validación temporal más robusta (walk-forward).
3. **Enriquecer variables:** incorporar datos climáticos reales por campaña, disponibilidad de stock y características del catálogo provincial.
4. **Modelos probabilísticos:** reemplazar la alerta binaria por una estimación de probabilidad calibrada y intervalos de confianza.
5. **Optimización logística:** conectar la proyección con un modelo de asignación de inventario y de armado de kits.
6. **Integración institucional:** evaluar una base de datos compartida con el municipio y el INTA para actualizar los datos sin intervención manual.
7. **Monitoreo en producción:** registrar predicciones y resultados reales para medir deriva (drift) y reentrenar con evidencia.

9. Referencias

- Documentación oficial de Flask. *Flask Documentation*. https://flask.palletsprojects.com/
- Documentación oficial de scikit-learn. *scikit-learn: Machine Learning in Python*. https://scikit-learn.org/stable/
- Documentación oficial de pandas. *pandas: Python Data Analysis Library*. https://pandas.pydata.org/docs/
- Documentación oficial de NumPy. *NumPy Reference*. https://numpy.org/doc/
- Documentación oficial de deck.gl. *deck.gl: WebGL-powered visualization*. https://deck.gl/docs
- Documentación oficial de Chart.js. *Chart.js Documentation*. https://www.chartjs.org/docs/
- Documentación oficial de Vercel. *Vercel Python Runtime*. https://vercel.com/docs
- INTA. *Programa ProHuerta*. https://www.argentina.gob.ar/inta/prohuerta
- Municipalidad de Chascomús. *Banco de Intercambio de Semillas*. https://www.chascomus.gob.ar/

Anexos

Anexo A. Estructura de archivos generados

Archivo	Contenido
datos/preparados/solicitudes_v2.csv	Una fila por solicitud, con targets, categorías y flags de calidad.
datos/preparados/solicitud_especies.csv	Una fila por solicitud y especie observada (2024–2026).
datos/preparados/solicitud_especies_imputada.csv	Canasta imputada para kits genéricos 2022–2023.
datos/preparados/solicitud_especies_completa.csv	Unión de especies observadas e imputadas.
datos/preparados/canasta_oi_referencia.csv	Ranking de especies OI y porcentaje de solicitudes.
datos/preparados/catalogo_temporada.csv	Oferta de catálogo y valor del sobre por temporada.
datos/preparados/dim_barrios.csv	Barrios, conteos, coordenadas y años con datos.
informe/auditoria_datos.md	Auditoría de fuentes, reconciliación y calidad.
informe/diccionario_datos.md	Diccionario completo de variables y fórmulas.
informe/comparacion_regresion.csv	Comparación de algoritmos de regresión.
informe/comparacion_clasificacion.csv	Comparación de configuraciones del clasificador.
informe/predicciones_test.csv	Predicciones sobre el conjunto de prueba.

Tabla A1. Archivos principales generados por el pipeline y el entrenamiento.

Anexo B. Reproducibilidad

Los pasos para reproducir el proyecto desde cero son los siguientes:

1. Crear un entorno virtual e instalar dependencias: **pip install -r requirements.txt**.
2. Regenerar los datos preparados: **python preparar_datos.py**.
3. Reentrenar los modelos de stock: **python entrenar_guardar_modelos.py**.
4. Reentrenar el modelo climático purgado: **python act_lluvia.py**.
5. Iniciar la aplicación: **python app.py** y abrir **http://localhost:5000**.

El proyecto también puede desplegarse en Vercel conectando el repositorio. El archivo **vercel.json** ya contiene la configuración necesaria.

Anexo C. Glosario

Término	Definición
Sobre equivalente	Unidad que representa una variedad de semilla solicitada.
Cobertura del catálogo	Porcentaje de variedades del catálogo que solicita una persona.
Alta demanda	Cobertura igual o superior al 75 % del catálogo de la temporada.
Baseline	Referencia histórica de demanda por barrio y temporada, ajustada por crecimiento.
Sobredemanda	Proyección que supera el baseline en al menos 10 % y 10 sobres.
Muestra limitada	Barrio con menos de 10 solicitudes o menos de 2 años de datos.
Data leakage	Uso de información no disponible al momento de predecir, que infla el desempeño.
Pipeline	Secuencia reproducible de preprocesamiento y modelo de scikit-learn.
Escenario base	Proyección construida con supuestos explícitos, no un pronóstico validado.

Tabla C1. Glosario de términos utilizados en el informe.

Nota de entrega. Este informe se complementa con el repositorio del proyecto, la aplicación desplegada y los archivos de auditoría y métricas citados. Los campos **Docente** y **Comisión** de la portada deben completarse antes de la entrega final.